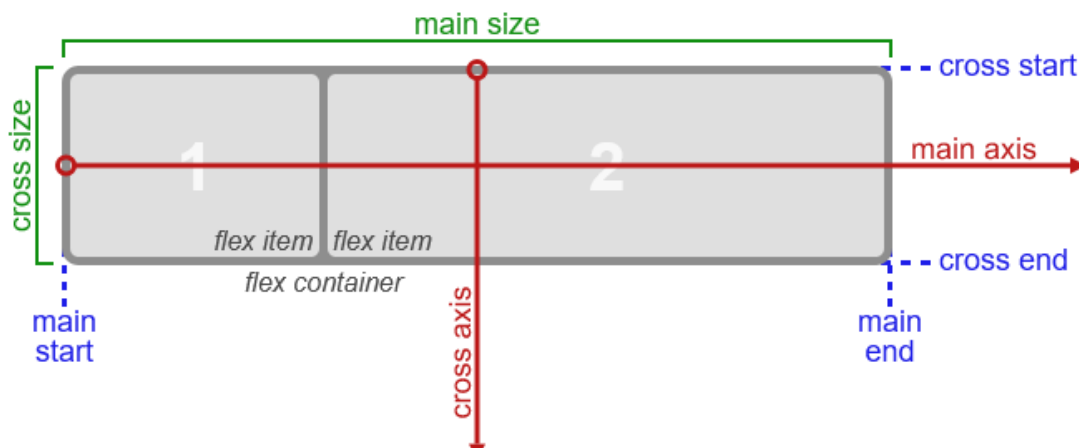


Laboratorio #2 CSS Adaptable

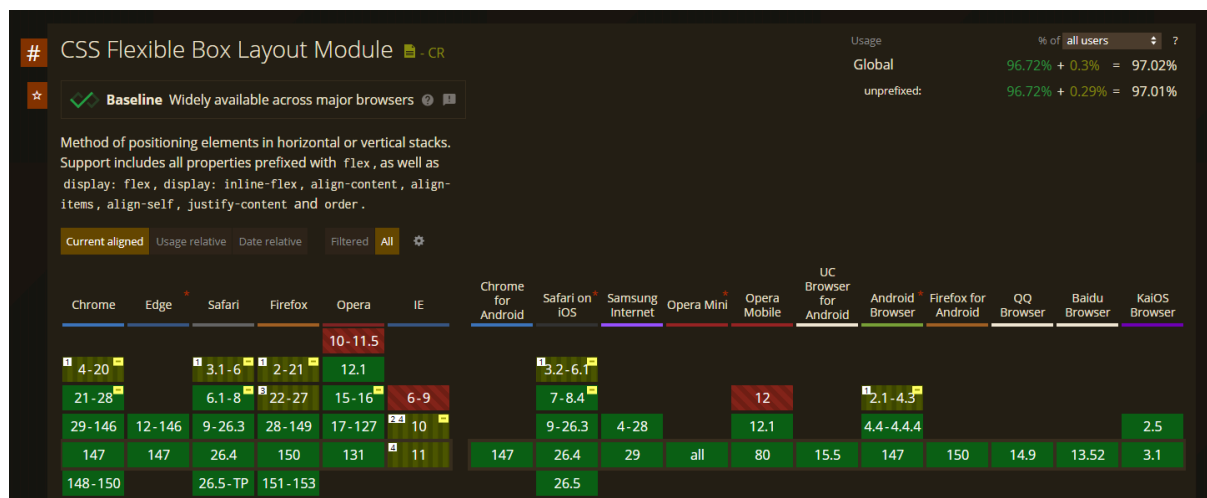
Flexbox

Flexbox es un método de diseño de página unidimensional para compaginar elementos en filas o columnas. Los elementos de contenido se ensanchan para rellenar el espacio adicional y se encogen para caber en espacios más pequeños. Se basa en un contenedor flexible (flex container), que a su vez contiene varios elementos flexibles (flex items). El contenedor otorga sus propiedades a los elementos, es decir: los elementos o flexboxes deben su flexibilidad al hecho de estar dentro del contenedor.

Más adelante veremos cómo manipular las características, tamaño y alineación tanto del contenedor flexible como de sus elementos interiores. No obstante, es importante conocer los términos de tamaño y distintas direcciones aplicados a un contenedor flexible (El siguiente diagrama es para un flex container en fila).



Flexbox CSS es soportado por la gran mayoría de los navegadores más importantes tanto en desktop como en mobile. La única excepción es el soporte parcial que ofrece Internet Explorer en su última versión IE11. No obstante, IE es un navegador actualmente en desuso, siendo su última versión lanzada en el 2013 y solo ocupando el 0.29% de la base de usuarios en la actualidad.



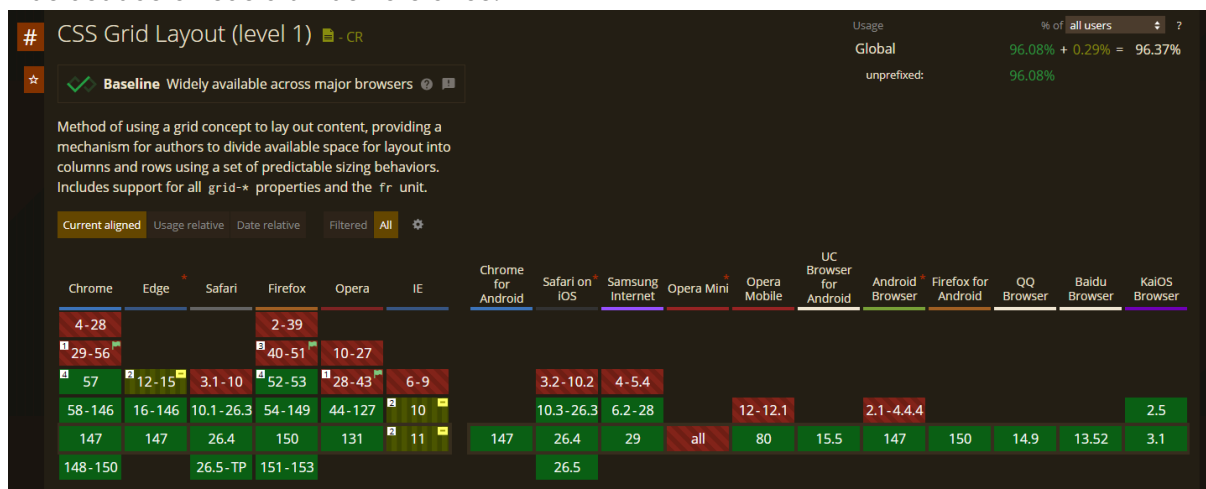
Captura obtenida de <https://caniuse.com/flexbox> el 22 de abril del 2026.

Grid CSS

CSS grid layout o CSS grid es una técnica de las Hojas de Estilo en Cascada que permite a los desarrolladores web crear diseños complejos y adaptables con mayor facilidad en todos los navegadores. A diferencia de Flexbox que se encuentra limitado a una sola dimensión, Grid permite manejar diseños bidimensionales.

La propiedad CSS grid es un [shorthand](#) que permite definir todas las propiedades grid explícitas ([grid-template-rows](#), [grid-template-columns](#), y [grid-template-areas](#)), implícitas ([grid-auto-rows](#), [grid-auto-columns](#), y [grid-auto-flow](#)), y relativas a gutter ([grid-column-gap](#) y [grid-row-gap](#)) en una sola declaración.

Al igual que Flexbox, Grid CSS es soportado por todos los navegadores más importantes y más usados en sus últimas versiones.



Captura obtenida de <https://caniuse.com/css-grid> el 22 de abril del 2026.

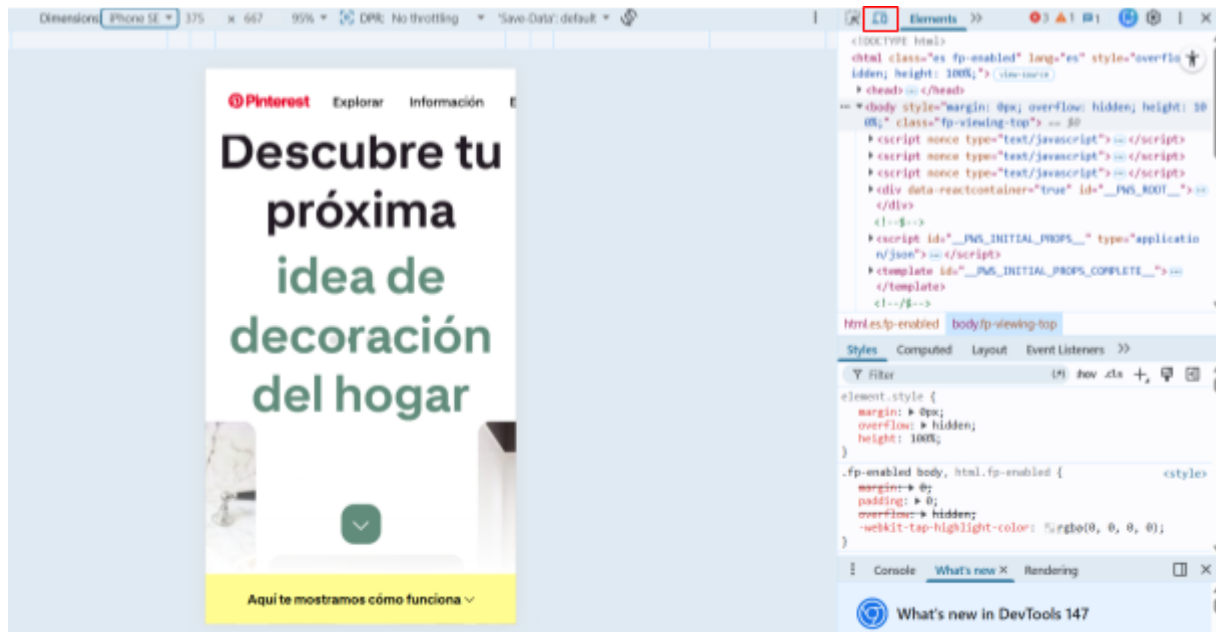
Herramientas para desarrolladores y diseños adaptables

Las herramientas para desarrolladores de la mayoría de navegadores nos permiten simular tamaños de pantallas pequeños (dispositivos móviles). Esta opción se activa de manera similar en la mayoría de navegadores, veamos 3 navegadores de ejemplo:

- Para los tres navegadores, oprime la tecla F12 dentro de la página para abrir las herramientas de desarrollador. Al hacer esto, aparecerá una nueva ventana a la derecha, izquierda, abajo o como una ventana flotante dependiendo de la configuración de layout que tengas en tu navegador.

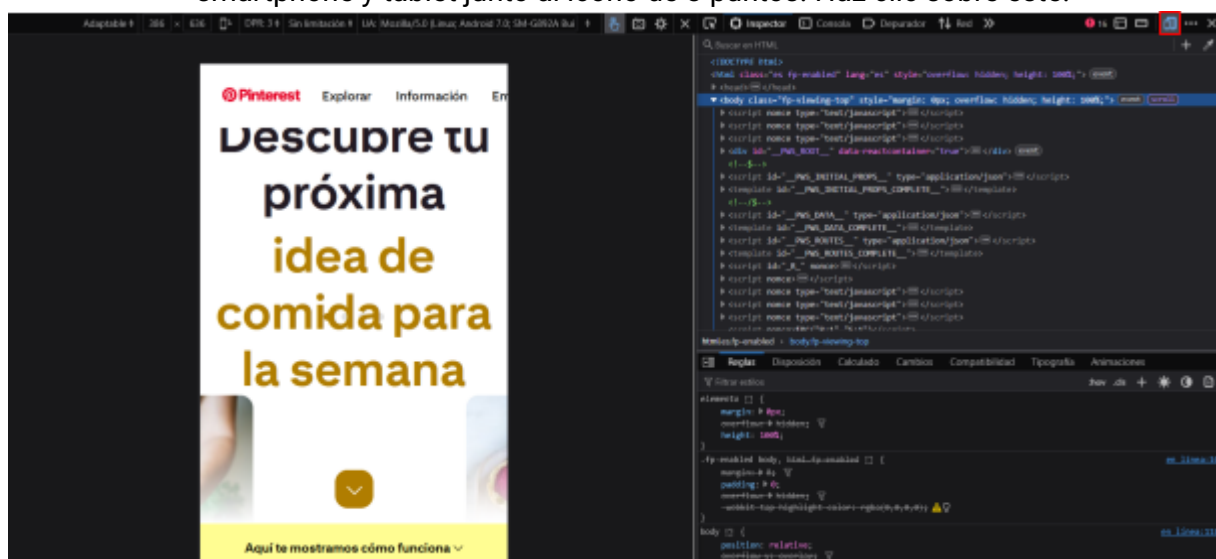
Google Chrome:

- En la esquina superior izquierda de la nueva ventana hay dos iconos pequeños. Haz clic en el segundo ícono que tiene forma de una laptop y smartphone.



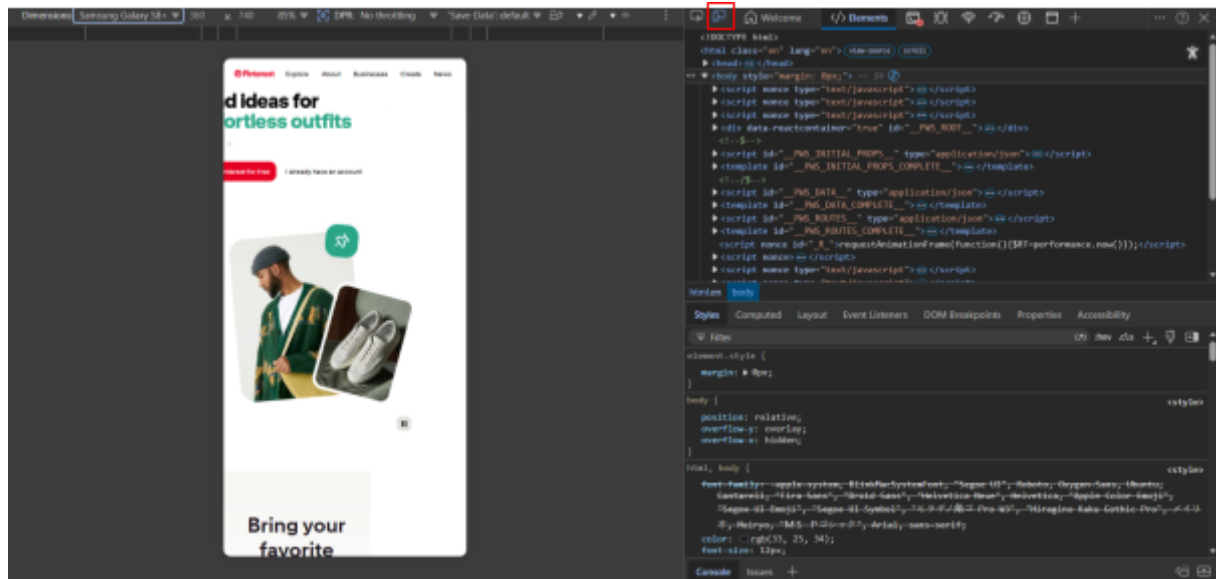
Firefox:

- En la esquina superior derecha de la nueva ventana ubica el ícono del smartphone y tablet junto al icono de 3 puntos. Haz clic sobre este.



Microsoft Edge:

- En la esquina superior izquierda de la nueva ventana haz clic sobre el segundo ícono con forma de un smartphone y una tablet.



- Una vez activada la simulación de dispositivos, el tamaño del viewport de la página web tomará las dimensiones que se especifican en la barra superior de esta ventana.
- Estas dimensiones se pueden cambiar libremente si se usa la opción "Responsive" o "Adaptable", aunque también se pueden probar tamaños estándar de dispositivos móviles populares en cualquiera de los 3 navegadores vistos.

Ejercicios

- Práctica con Flexbox. Para poder probar todas las posibilidades que ofrece Flexbox CSS, vamos a:

- Crear una capa `class="contenedor"` que hará de padre, y siete capas `class="elemento"` numeradas.

```
<div class="contenedor">
  <div class="elemento">1</div>
  <div class="elemento">2</div>
  <div class="elemento">3</div>
  <div class="elemento">4</div>
  <div class="elemento">5</div>
  <div class="elemento">6</div>
  <div class="elemento">7</div>
</div>
```

- Vamos a colocar de inicio un tamaño de anchura del 25% para los elementos en relación al contenedor padre. Para comenzar a utilizar Flexbox añadimos al contenedor la propiedad `"display:flex"`

```
.contenedor{
  display:flex;
}
```

```
.elemento{  
    width:25%;  
}
```

El comportamiento de dirección y tamaño que tendrán los elementos de nuestro contenedor se adaptan a su padre, aunque hayamos definido una anchura de elementos del 25%, éstos ocuparan el 100% de anchura del contenedor entre la suma de todos. Por defecto, tiene ese comportamiento "flexible".

3. Veamos la propiedad "**flex-direction**", que puede tomar 4 valores y se aplica al padre (contenedor):
 - **flex-direction:row**; -> Los elementos se visualizan **de izquierda a derecha** (valor por defecto, similar al ejemplo 1)
 - **flex-direction:row-reverse**; -> Los elementos se visualizan **de derecha a izquierda**.
 - **flex-direction:column**; -> Los elementos se visualizan **de arriba hacia abajo**.
 - **flex-direction:column-reverse**; -> Los elementos se visualizan **de abajo hacia arriba**.

```
.contenedor{  
    display:flex;  
    flex-direction:row-reverse;  
}
```

Los elementos invierten su orden de visualización de derecha a izquierda, sin tener en cuenta el orden de maquetación.

4. Vamos a asignar a la propiedad "**flex-direction**" el valor column.

```
.contenedor{  
    display:flex;  
    flex-direction:column;  
}
```

Se visualizan formando una columna de arriba hacia abajo. En este caso, como los elementos miden el 25% del contenedor, su anchura sí toma el valor indicado. Si eliminamos la anchura, las capas se ajustarán al tamaño del contenedor, ocupando el 100% de anchura.

5. Existen otras propiedades de alineación para Flexbox como **flex-wrap**, **justify-content**, **align-items**, **align-self** y **align-content**. En <https://www.w3.org/TR/css-flexbox-1/> podrá visualizar estos y otros atributos con ejemplos para flexbox.

b) Práctica con Grid.

1. Cree un archivo HTML:

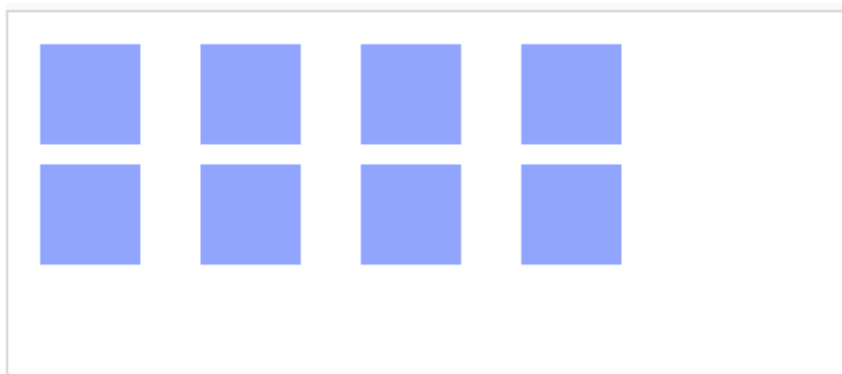
```
<div id="container">
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
</div>
```

2. Agregue las siguientes reglas CSS:

```
#container{
    display: grid;
    grid: repeat(2, 60px) / auto-flow 80px;
}

#container > div {
    background-color: #8ca0ff;
    width: 50px;
    height: 50px;
}
```

3. Analice el resultado:



- c) Práctica con media queries. Los media queries nos permiten aplicar reglas de estilo dependiendo de una o más características técnicas del dispositivo que solicita la visualización de la página web.

1. Cree un archivo HTML con la siguiente estructura:

```
<h1>Ejemplo de Media Query #1</h1>
<p>Reduce el ancho de la ventana a 600 px o menos.</p>
<p>Observe cómo el color de fondo cambió de blanco a azul.</p>
```

2. Cambie el color de fondo de la página a azul para dispositivos pequeños:

```
body {  
  text-align: center;  
}  
  
p {  
  font-size: 1.5rem;  
}  
  
@media (max-width: 600px) {  
  body {  
    background-color: #87ceeb;  
  }  
}
```

Es importante que el Media Query se ubique siempre al final del archivo CSS debido al orden de aplicación de las reglas de estilo.

3. Cambie el color de fondo de azul a rojo si el dispositivo tiene un ancho de entre 600 y 768 px. Podemos usar el operador and para lograr esto:

```
body {  
  text-align: center;  
  background-color: #87ceeb;  
}  
  
p {  
  font-size: 1.5rem;  
}  
  
@media (min-width: 600px) and (max-width: 768px) {  
  body {  
    background-color: #de3163;  
  }  
}
```

4. Para conocer otros atributos de Media Query puede consultar https://developer.mozilla.org/es/docs/Web/CSS/CSS_media_queries/Using_media_queries.

d) Práctica con multimedia flexible.

1. Realice una captura de pantalla, guarda la imagen con formato png (imagen1.png).
2. Use cualquier editor de imagen para cambiar el tamaño de la imagen, guardar la copia editada como imagen2.gif.

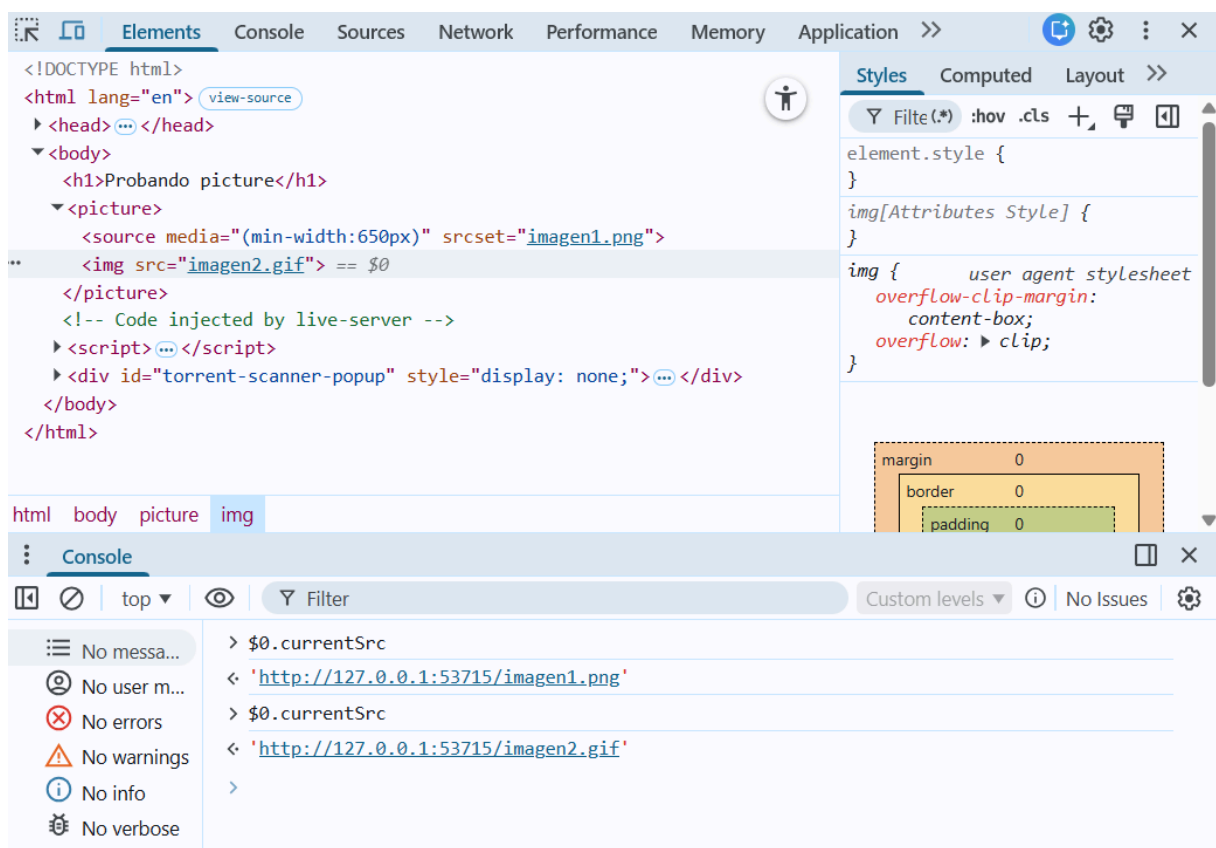
3. Crear un archivo HTML:

```
<h1>Probando picture</h1>
<picture>
  <source media="(min-width:650px)"
    srcset="imagen1.png">
  
</picture>
```

La etiqueta picture puede contener múltiples sources para distintos media queries. Sin embargo, para mayor compatibilidad entre los navegadores es necesario que incluya una etiqueta img al final que sirve como valor por defecto.

4. Usando el navegador, abra el HTML.

5. Para verificar la fuente actual de la imagen, haga clic derecho sobre la imagen y abra el inspector. En la consola de las propiedades DOM de la imagen escriba `$0.currentSrc`. Cambie el ancho del viewport a menos de 650px y escriba nuevamente el comando



Opcionalmente, se puede verificar el nombre de la imagen que se carga en la ventana de red.

e) Práctica con fuentes tipográficas flexibles.

1. Cree un archivo HTML con la siguiente estructura dentro de `<body>`

```
<h1>Probando tamaños de fuentes en tag p</h1>
<p class="p1">Soy rem</p>
<p class="p2">Soy em</p>
<p class="p3">Soy px </p>
<p class="p4">Soy porcentaje</p>
<p class="p5">Soy px </p>
```

2. Agregue las siguientes reglas CSS a las clases de los elementos HTML

```
body {
  text-align: center;
  background-color: #87ceeb;
}

.p1 {
  font-size: 1.5rem;
}

.p2 {
  font-size: 2.65em;
}

.p3 {
  font-size: 2.65ex;
}

.p4 {
  font-size: 25%;
}

.p5 {
  font-size: 4px;
}
```

3. Identifique las diferencias entre estas definiciones. En la práctica podrían parecer equivalentes pero:

- **em** es relativo al tamaño de fuente del elemento padre, por lo que si se busca escalar el tamaño del elemento en función del tamaño de su padre, se utiliza **em**.
- **rem** es relativo al tamaño de fuente de la raíz (HTML) y si este no está establecido se usará el que tenga configurado el navegador. Por lo que si se desea escalar el tamaño del elemento en función del tamaño de la raíz, sin importar cuál sea el tamaño principal, usa **rem**. Si has



utilizado **em** y estás encontrando problemas de tamaño debido a muchos elementos anidados, **rem** sea probablemente la mejor opción.

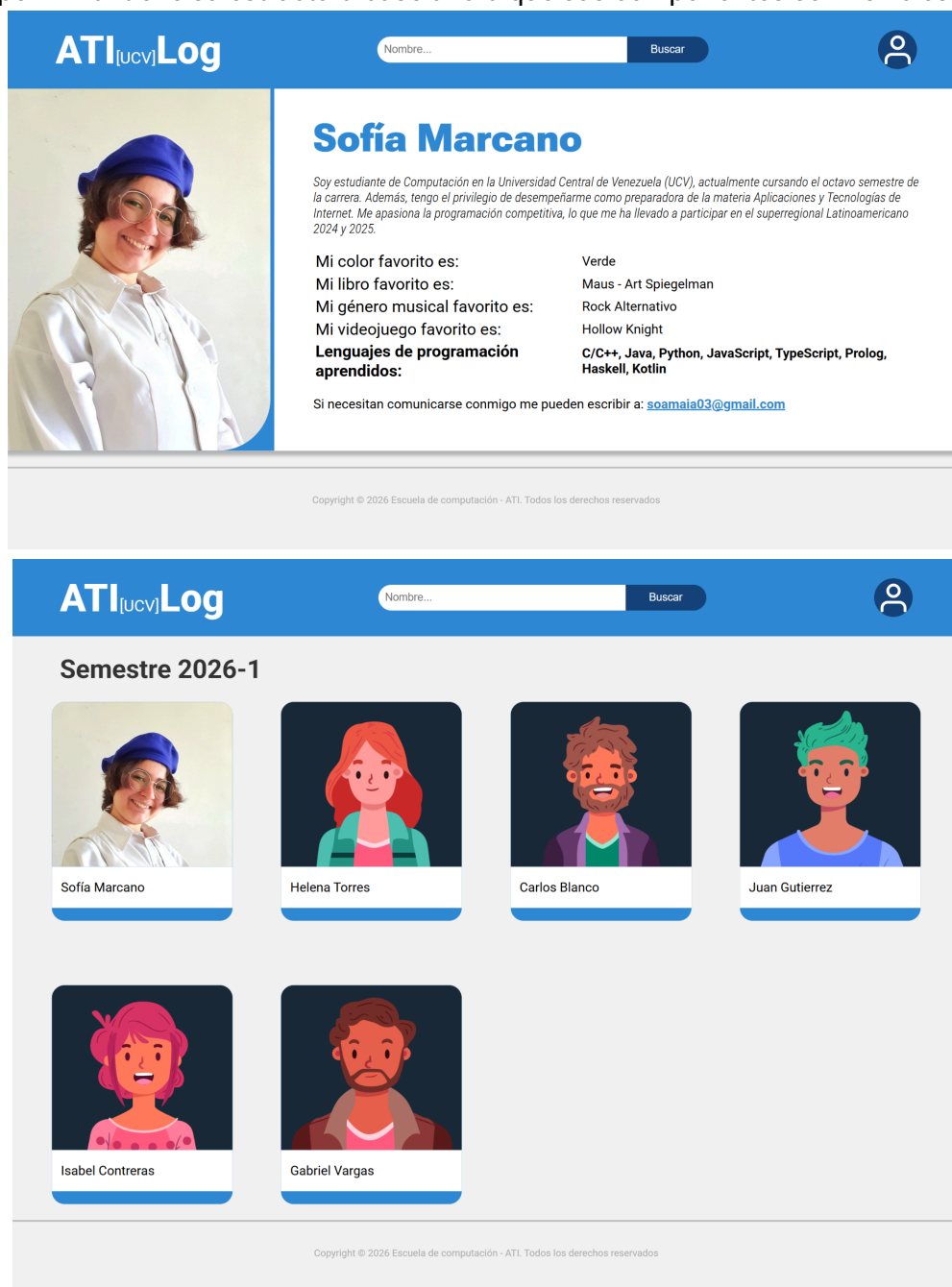
- % es un porcentaje del ancho o alto del elemento padre. Las unidades de porcentaje suelen ser útiles para establecer el ancho de los márgenes.

Reto 2 (at home)

Antes de empezar, verifique que haya hecho *merge* de la rama del reto anterior a la rama main de su repositorio. Luego, crea una nueva rama a partir del main que tenga el nombre "reto-2". Trabaje este nuevo reto sobre esa nueva rama.

Crear CSS responsive para las páginas web creadas en el laboratorio previo.

1. Actualice el archivo `style.css` creado en el laboratorio anterior, configurando las características de las columnas y filas de la página de perfil usando flexbox y grid. Aprovecha de ajustar tamaños de componentes, márgenes o padding para lograr una mejor distribución de los elementos en la página. Asegúrese de que la página de perfil mantiene su estructura base ahora que sus componentes son flexibles:



2. Chequee el llamado desde los archivos HTML a las nuevas referencias, verifique que todo lo asociado al diseño de las páginas se está manejando desde el archivo `styles.css` y que no existe CSS incrustado en ninguno de los dos archivos HTML.
3. Actualice el CSS configurando las características de cuatro tipos de pantallas usando media queries con breakpoints:
 - 320px - 480px: Dispositivos móviles
 - 481px - 768px: iPads, tablets
 - 769px - 1024px: pantallas pequeñas, ordenadores portátiles
 - 1025px - 1200px: pantallas grandes, ordenadores de escritorios

Para las pantallas de dispositivos pequeños use las siguientes imágenes de guía:



Note que, en el header se remplazan tanto la barra de búsqueda como el botón de perfil por un menú. No es necesario hacer este menú desplegable funcional para este reto.

Para los tamaños intermedios y grandes adapte el diseño ya conocido.

4. Renombre la imagen de la foto de perfil a {CEDULA}Big.jpg (ej: 29765263Big.jpg), realice una copia con el nombre {CEDULA}Small.jpg y actualice la imagen reduciendo su tamaño.
5. Establezca en el CSS, usando media queries, que la aplicación al realizar responsive use la imagen pequeña para teléfonos móviles y tablets, mientras que en la versión para ordenadores se cambie a la imagen grande.
6. Incorpore el atributo font-size a las etiquetas usadas en el HTML para definir las unidades de medida de las fuentes tipográficas de forma flexible, en base al análisis y conclusiones del ejercicio e.
7. Empleando lo aprendido sobre las herramientas de desarrolladores de los navegadores y los estilos de pantallas, verifique que se aplican correctamente los estilos a cada tamaño de pantalla previamente descrito.
8. Tome una captura de pantalla por cada tipo de pantalla, asegúrese que en la captura sea visible el nombre de la imagen que usa la página en las herramientas de desarrolladores.
9. Repita los pasos del 5 al 8 para la página del listado.
10. Al tener todo listo, haga commit del proyecto con el mensaje “versión responsive”.

Al terminar, colocar en el reto del Aula Virtual el url de la rama del repositorio de GitHub que uso para hacer este ejercicio junto a las imágenes de las pruebas del paso 8.